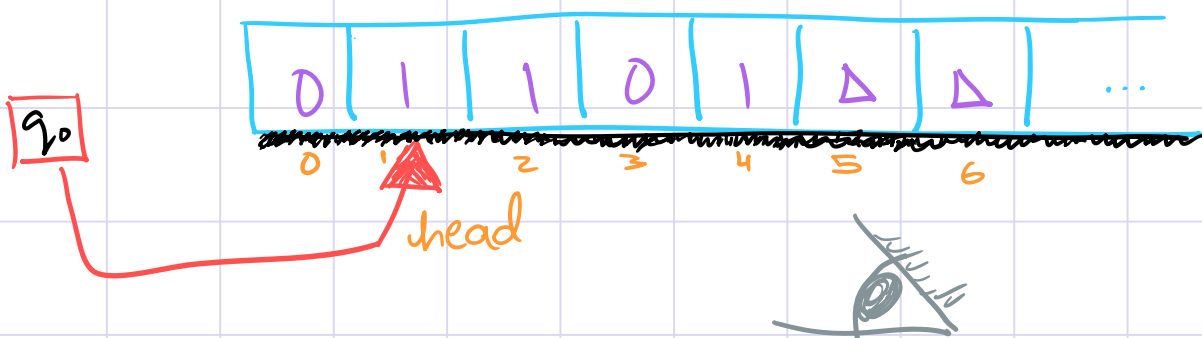


Oblivious Turing Machines

Data Oblivious Algorithms

Eavesdropper may have access to which memory blocks are being accessed.

Not good for computer security!



This eavesdropper might be able to figure out certain properties of a private key with these memory accesses.

In a data oblivious algorithm the memory access pattern will be the same for all inputs of the same size.

In a data oblivious algorithm, you cannot learn anything about the input from the way the memory is being accessed.

What would it look like for Turing Machines?
 $\delta(?) = ?$

In an "oblivious" Turing Machine, the tape head movement (i.e. $\{L, R\}$) will depend only on the size of the input $|x|$ and not the input itself $|x| \mid x_1 \mid x_2 \mid x_3 \mid \dots$

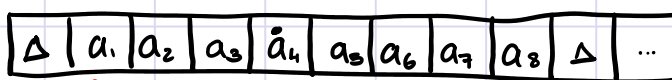
Are Oblivious Turing Machines equivalent to ordinary, single tape Turing Machines?

How would you construct an Oblivious TM?

Try to simulate an ordinary TM M_0 , on an oblivious TM M

Rough idea -

M



Sweep 1 $\longrightarrow \longrightarrow \longrightarrow \longrightarrow$

If we are sweeping from left to right and if $\delta_0(\cdot)$ required the tape head to move "right," then we move the virtual head to the right.

If δ_0 required us to move the tape head to the left, then we will move the oblivious TM's head all the way to the right, and start sweeping from right to left. Then we will update the virtual head.

Sweep 2 $\longleftarrow \longleftarrow \longleftarrow \longleftarrow$

$$T_0 = \{0, 1, \Delta\}$$

$$\text{for } i = 1, 2, 3, \dots, a_i \in \{0, 1\}$$

Augment T_0 to get

$$T = \{0, 0, 1, 1, \Delta\}$$

$$M_0 = \langle Q_0, \Sigma, T_0, \delta_0 \rangle$$

$$T_0, \delta_0$$

$$q_0, \text{start}, q_0, \text{accept}, q_0, \text{reject}$$

$$M = \langle Q, \Sigma, T, \delta, q_0, q_{\text{accept}}, q_{\text{reject}} \rangle$$

Theorem 1: For any ordinary single tape TM M_0 that decides a language L in time $T_{M_0}(n)$, there exists an oblivious TM M that decides the same language in time $O(T_{M_0}(n)^2)$.

Proof:

Lemma 1: The Oblivious TM model is equivalent to the ordinary single tape TM model

PROOF OF LEMMA 1

($\Rightarrow$) Given an ordinary, single tape TM M_0 , we can construct an equivalent Oblivious TM M as follows,

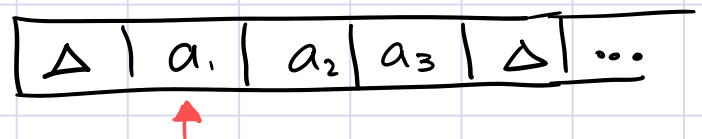
Let $M_0 = \langle Q_0, \Sigma_0, T_0, \delta_0, q_{0,start}, q_{0,accept}, q_{0,reject} \rangle$

Construct $M = \langle Q, \Sigma, T, \delta, q_{start}, q_{accept}, q_{reject} \rangle$

Here the input alphabet stays the same. $\Sigma = \Sigma_0$

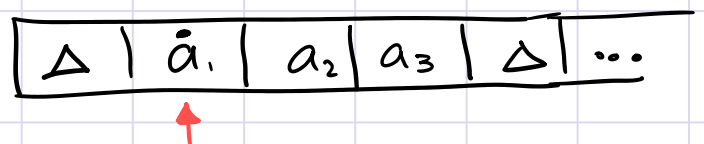
We create T st. it includes every symbol in T_0 and for any ~~non-blank~~ symbol $a \in T_0$, we add a symbol $\bar{a}$, which acts as a "virtual" head.

If M_0 starts like this:



$a_i \in T_0$ for M_0

The M will start like:



$a_i \in T$ for M

Every time M points to a virtual head, we will "execute" S_0 of M_0 . Initially, M is pointing to the first cell, which contains the virtual head. M will have

$$S(q_{\text{start}}, \bar{a}) = (q_{k_1}, b, R) \quad \text{if } M_0 \text{ had } S_0(q_{\text{start}}, a) = (q_{k_2}, b, R)$$

$$S(q_{k_1}, c) = (q_{k_2}, \bar{c}, R)$$



Now M 's head will go right without changing the tape, until it reaches the end. Once it encounters Δ , the tape head starts sweeping from right to left.

M will keep going left without changing the tape symbols, till it reaches the virtual head. Then, M will execute the relevant transition from S_0 .

Now it is enough to show how M will simulate an arbitrary transition of M_0 .

We incorporate S_0 into S by using the virtual head.

case I : $\delta_0(q_i, a) = (q_j, b, R)$ and M is
Sweeping from left to right. (starting from the first cell)

As before, keep moving head to the right till you encounter the virtual head.

$$\text{execute } \delta(q_i, a) = (q_j, b, R)$$

$$\delta(q_i, x) = (q_j, x, R) \quad \forall x \in T$$

Then, continue sweeping right till Δ is encountered.

case II : $\delta_0(q_i, a) = (q_j, b, R)$ and
 M is sweeping from right to left.
(starting from the end of the string on the tape).

Keep moving the head to the left till you encounter the virtual head.

$$\text{execute } \delta(q_i, a) = (q_j, b, \underline{L})$$

Keep moving the head to the left till you reach Δ
Start sweeping from left to right till you reach the virtual head.

$$\text{execute } \delta(q_{k_1}, b) = (q_{k_2}, b, R)$$

$$\delta(q_{k_2}, x) = (q_j, x, R) \quad \forall x \in T$$

Continue sweeping from left to right.

case III : $\delta_0(q_i, a) = (q_j, b, \underline{L})$ and M is
Sweeping from left to right. (starting from the first cell)

Keep moving the head to the right till you encounter the virtual head.

Execute $\delta(q_i, a) = (q_i, b, \underline{R})$

Keep moving the head to the right till you reach Δ
Start sweeping from right to left till you reach the virtual head.

Execute $\delta(q_{k_1}, b) = (q_{k_2}, b, \underline{L})$
 $\delta(q_{k_2}, x) = (q_j, x, \underline{L}) \quad \forall x \in T \setminus \{\Delta\}$

Continue sweeping from right to left.

case IV : $\delta_0(q_i, a) = (q_j, b, \underline{L})$ and

M is sweeping from right to left.
(starting from the end of the string on the tape).

As before, keep moving head to the left till you encounter the virtual head.

execute $\delta(q_i, a) = (q_i, b, \underline{L})$
 $\delta(q_i, x) = (q_j, x, \underline{L}) \quad \forall x \in T \setminus \{\Delta\}$

Then, continue sweeping left till Δ is encountered.

All the previous δ transitions included the virtual head. The case in which the virtual head is not present in $\delta(,) = (, ,)$ is when the tape head is sweeping either from left to right, or from right to left.

For each case we assign a state: $q_{\text{sweep left}}$, $q_{\text{sweep right}}$.

So we have,

$$\delta(q_{\text{sweep left}}, x) = (q_{\text{sweep left}}, x, L) \quad \forall x \in T \setminus \{\Delta\}$$

$$\delta(q_{\text{sweep right}}, x) = (q_{\text{sweep right}}, x, R) \quad \forall x \in T \setminus \{\Delta\}$$

Handling blank symbols:

We assume that the tape starts with a blank symbol, followed by the input string. Also, all cells after the input contain Δ .

If the first and last cells (after and before Δ) are not the virtual head, then M changes direction of the sweep without any extra steps.

$$\delta(q_{\text{sweep left}}, \Delta) = (q_{\text{sweep right}}, \Delta, R)$$

$$\delta(q_{\text{sweep right}}, \Delta) = (q_{\text{sweep left}}, \Delta, L)$$

The problem occurs when Non blank

case (a): The first symbol is a virtual head
and M_0 does $\delta(q_i, a) = (q_j, b, L)$

Here we're moving from right to left.

execute: $\delta(q_i, a) = (q_j, b, L)$

$\delta(q_i, \Delta) = (q_j, \Delta, R)$

Continue sweeping from right to left.

Basically, M is the TM model with a single tape,
which is infinite only from the right hand side
(You can use the 2-sided infinite tape model as)
well since these are equivalent

case (b): The first blank symbol after the non-blank symbols
has the virtual head.

Δ	$x_1 x_2 \dots x_n$	Δ	Δ	Δ	$\dots$
----------	---------------------	----------	----------	----------	---------

But this case is already handled by
case (I), case (II)!

Here we are allowing for blanks on the RHS of
the non-blank symbols to have the virtual head.

This way M has access to the infinite tape

($\Leftarrow$) Given an Oblivious TM M , we can find an ordinary single tape TM M_0 that is equivalent to M .

This is trivially true!

$\therefore$ The oblivious TM model is computationally equivalent to the single tape TM model $\square$

Proof of theorem 1: (Analysis)

Given a TM M_0 , construct an oblivious TM M using Lemma 1

We know for input x , and $n = |x|$, M runs within time $T(n)$, i.e. - it takes ^{at most} $T(n)$ steps

(delta transitions) to finish its computation and halt.

This means that while processing x , M cannot reach cells beyond index $T(n)$. This bound applies to the oblivious TM as well, since it won't need more cells than M_0 took to process x .

In the worst case scenario, M 's ^{virtual} head is at the second non-blank symbol, and has to move it to the left, but M is sweeping from Left to Right.

So to execute one $S_0(\cdot)$, M has to go all the way to the right, hit the boundary (which is at most $T(n)$), and then come all the way back (Sweep from right to left).

$\therefore$ One step for M_0 takes at most $2T(n)$ steps for M

$\Rightarrow T(n)$ steps for M_0 will take at most $2T(n)^2 + c$ steps (for some constant overhead c) for M .

$\therefore M$'s running time is $O(T(n)^2)$ $\square$

